Visitor

Visitor · Pretty-print / evaluate expression AST · Build with Waqar

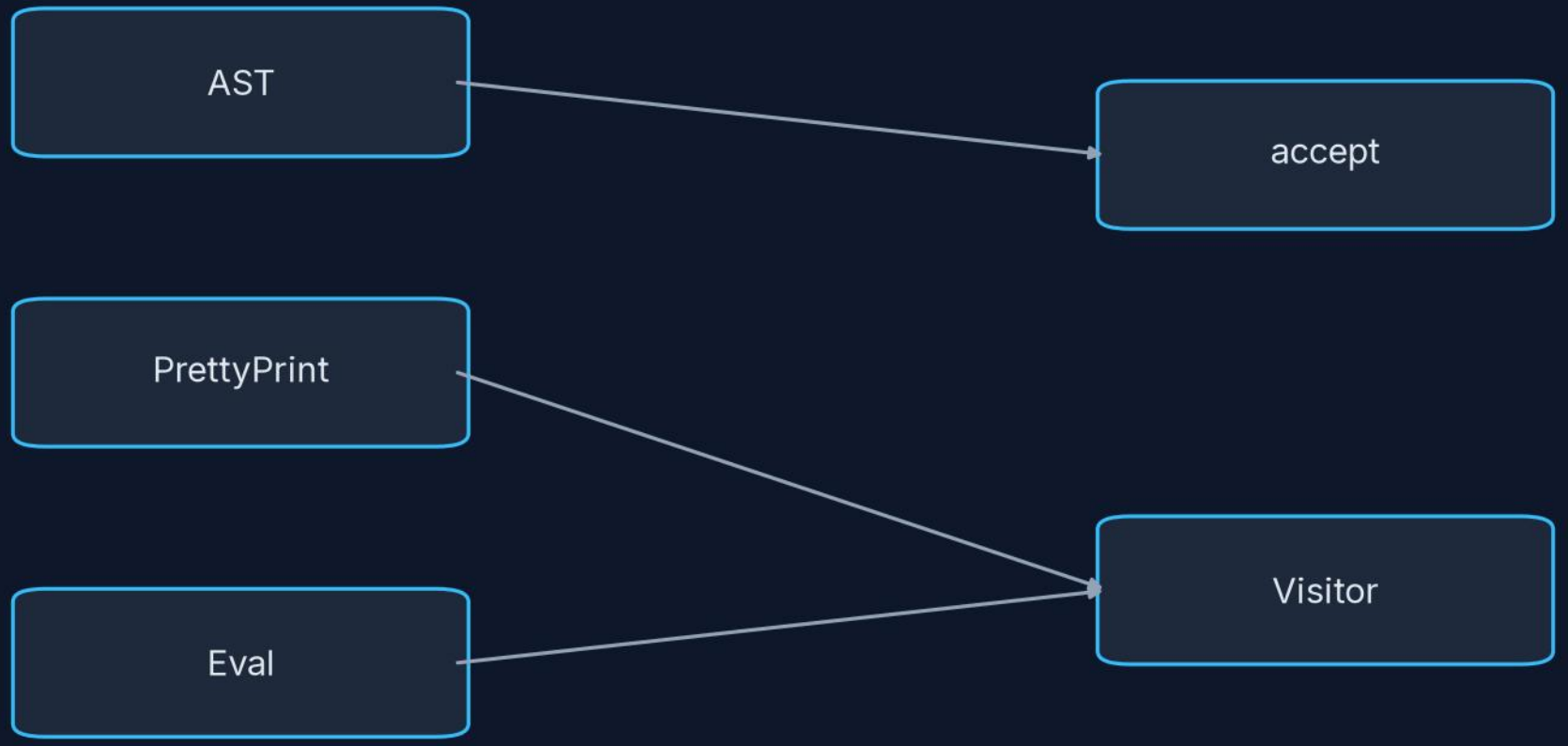
Today's Hook

- Problem: Pretty-print / evaluate expression AST
- Pattern: Visitor — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Pretty-print / evaluate expression AST
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Visitor: Pretty-print / evaluate expression AST

Real-World Scenario

- Running example: Pretty-print / evaluate expression AST
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.visitor;

import com.buildwithwaqar.patterns.gof.behavioral.visitor.*;
/** Problem: compute shape areas without modifying shape classes (or use sealed switch). */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Visitor problem: shape areas ===");
    Element c = new CircleEl(2);
    System.out.println("pm=" + ShapeVisitor.area(c));
    System.out.println("visitor=" + ShapeVisitor.areaViaVisitor(c));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.visitor;

sealed interface Element permits CircleEl, RectEl {
    <R> R accept(Visitor<R> visitor);
}

record CircleEl(double r) implements Element {
    public <R> R accept(Visitor<R> visitor) { return visitor.visit(this); }
}

record RectEl(double w, double h) implements Element {
    public <R> R accept(Visitor<R> visitor) { return visitor.visit(this); }
}

interface Visitor<R> {
    R visit(CircleEl c);
    R visit(RectEl r);
}

/** Visitor vs Java 25 pattern matching on sealed types. */
public final class ShapeVisitor {
    private ShapeVisitor() {}
```

Solution (2)

```
    public static double area(Element e) {
        return switch (e) {
            case CircleEl(var r) -> Math.PI * r * r;
            case RectEl(var w, var h) -> w * h;
        };
    }

    public static double areaViaVisitor(Element e) {
        return e.accept(new Visitor<>() {
            public Double visit(CircleEl c) { return Math.PI * c.r() * c.r(); }
            public Double visit(RectEl r) { return r.w() * r.h(); }
        });
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.visitor;

import com.buildwithwaqar.patterns.gof.behavioral.visitor.*;
/** Problem: compute shape areas without modifying shape classes (or use sealed switch). */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Visitor problem: shape areas ===");
    Element c = new CircleEl(2);
    System.out.println("pm=" + ShapeVisitor.area(c));
    System.out.println("visitor=" + ShapeVisitor.areaViaVisitor(c));
  }
}
```

Walkthrough Output

```
=== Visitor problem: shape areas ===
pm=12.566370614359172
visitor=12.566370614359172
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Visitor.

Related Patterns

- Contrast Visitor with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Visitor → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.